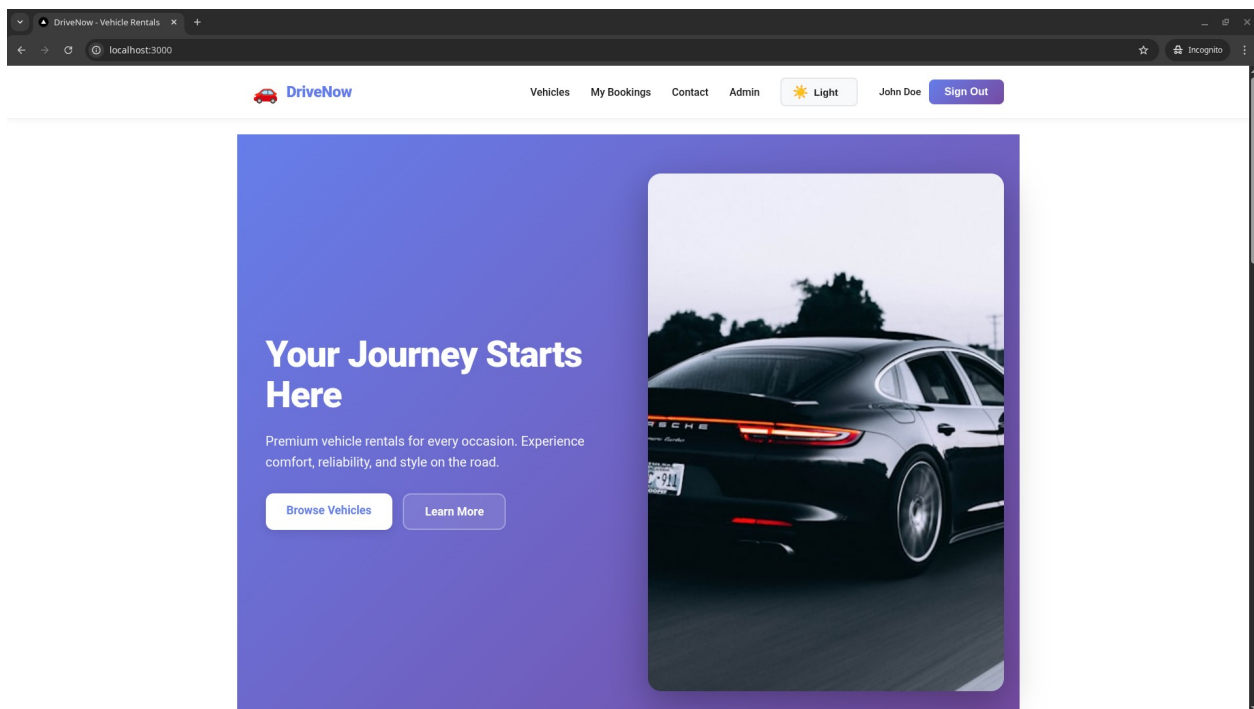
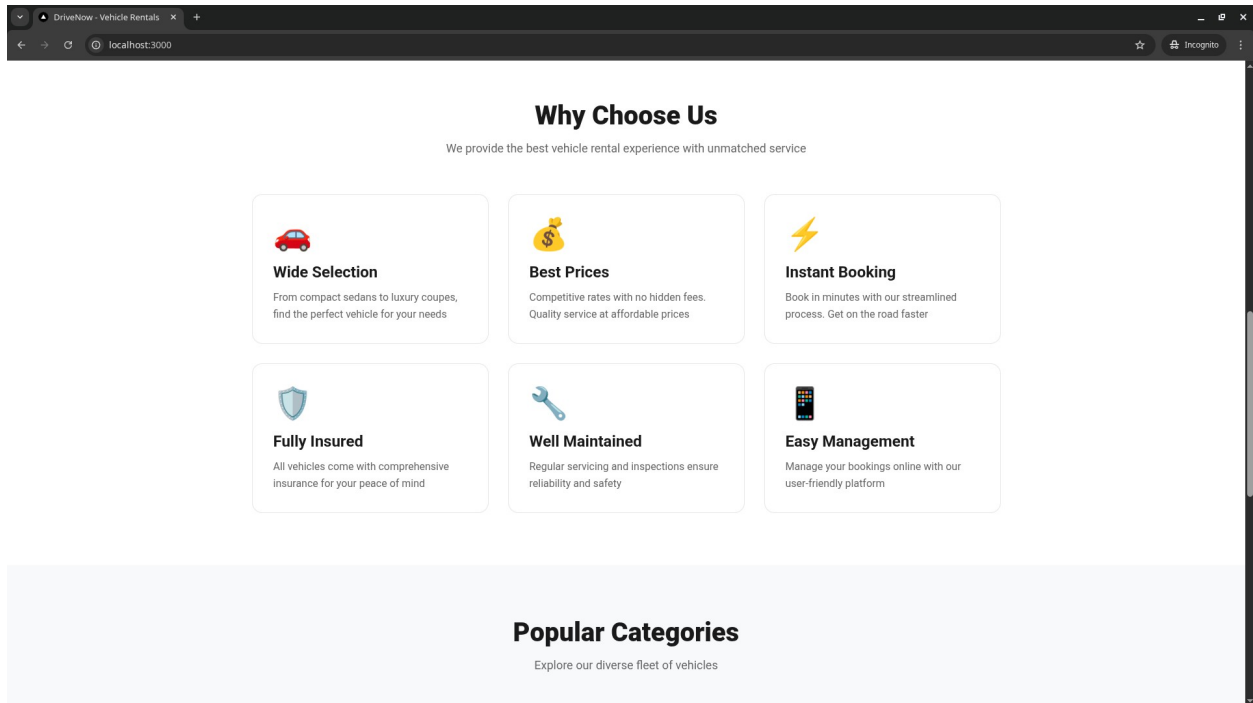


Vehicle Rental System

DBMS Project prepared by:

- Gopi Chand (2406002)
- Yaksh Bariya (2406025)
- Khairul Bashar (2406054)
- Himanshu (2406059)





A modern web application for renting vehicles online. Built with Next.js, React, MySQL, and TypeScript.

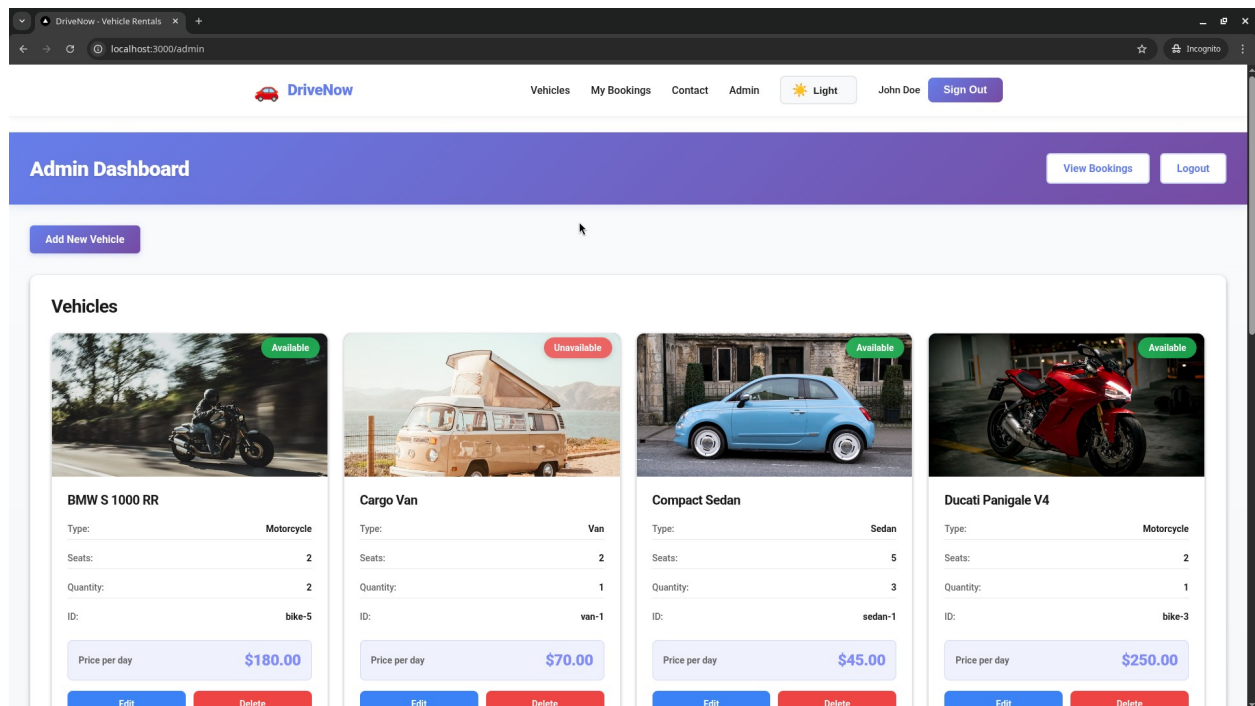
What This Project Does

Think of it as an online platform where customers can browse and book vehicles. The system has a user-friendly interface for browsing cars and an admin panel for managing inventory.

For Customers

- Browse all available vehicles
- Filter by type, price, and capacity
- Create an account and login
- Book a vehicle for specific dates
- Track booking history

Admin Login

[Login](#)

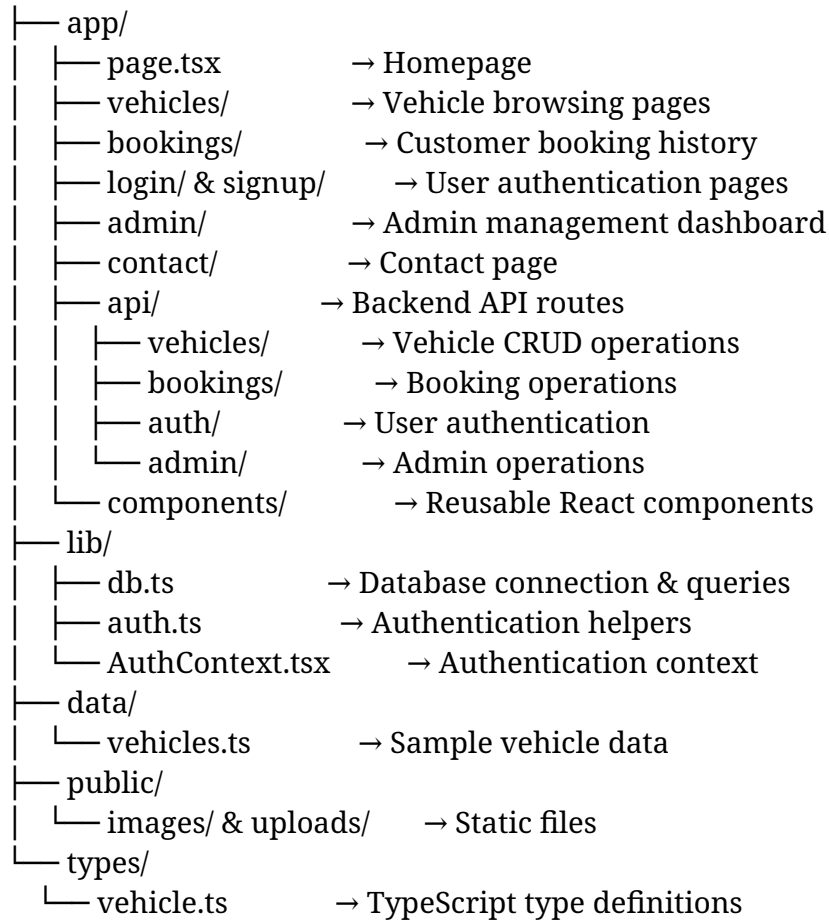
The screenshot shows the Admin Dashboard interface. At the top, there's a navigation bar with the DriveNow logo, links for Vehicles, My Bookings, Contact, and Admin, a Light mode toggle, and a user profile for John Doe with a Sign Out button. Below this is a purple header bar labeled "Admin Dashboard" with buttons for "View Bookings" and "Logout". A "Add New Vehicle" button is located on the left. The main content area is titled "Vehicles" and displays a grid of four vehicle cards. Each card includes a vehicle image, a status badge (Available or Unavailable), the vehicle name, type, seats, quantity, ID, and price per day. Each card also has "Edit" and "Delete" buttons.

Vehicle	Status	Type	Seats	Quantity	ID	Price per day
BMW S 1000 RR	Available	Motorcycle	2	2	bike-5	\$180.00
Cargo Van	Unavailable	Van	2	1	van-1	\$70.00
Compact Sedan	Available	Sedan	5	3	sedan-1	\$45.00
Ducati Panigale V4	Available	Motorcycle	2	1	bike-3	\$250.00

For Admins

- Login to the admin panel
- Add new vehicles to the system
- Edit or delete existing vehicles
- Manage vehicle availability
- Initialize the database with sample data

Project Structure



Technology Stack

- **Frontend:** Next.js 16, React 19, TypeScript
- **Backend:** Next.js API Routes
- **Database:** MySQL
- **Styling:** CSS Modules
- **Authentication:** Custom session-based (cookies)

Getting Started

Prerequisites

- Node.js (v24 or higher)
- MySQL server running locally or accessible
- yarn package manager

Installation

1. Install Node.js v24, MariaDB (MySQL).

2. Set up environment variables:

Create a .env.local file in the project root:

```
DB_HOST=localhost  
DB_PORT=3306  
DB_USER=root  
DB_PASSWORD=your_mysql_password  
DB_NAME=vehicle_rental
```

```
ADMIN_USERNAME=admin  
ADMIN_PASSWORD=admin123
```

3. Create the MySQL database:

4. Start the development server:

5. Visit the application:

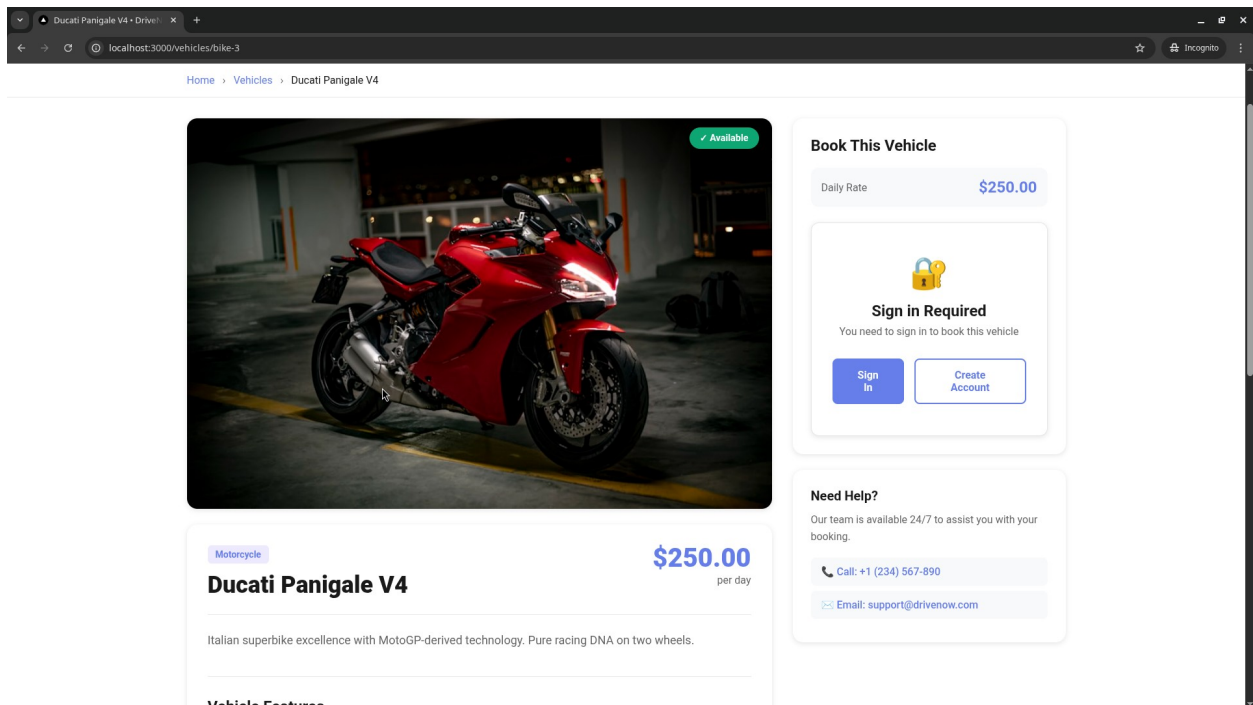
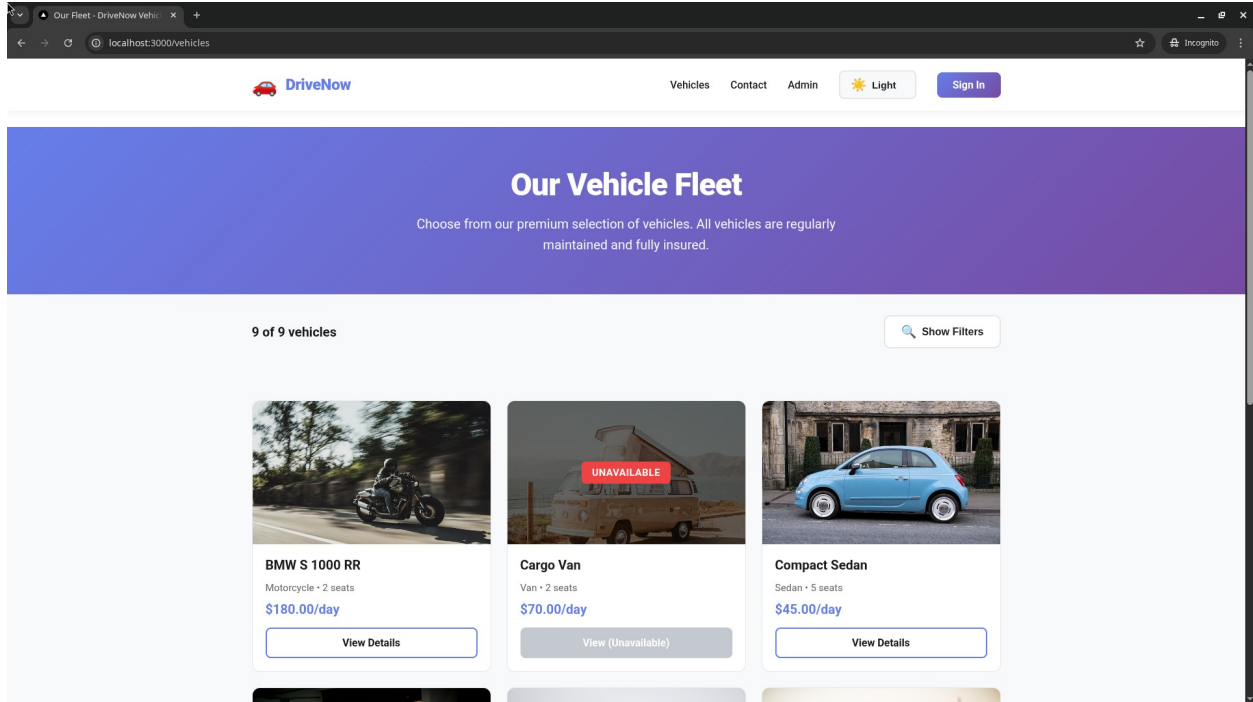
- o Homepage: <http://localhost:3000>
- o Admin Panel: <http://localhost:3000/admin>

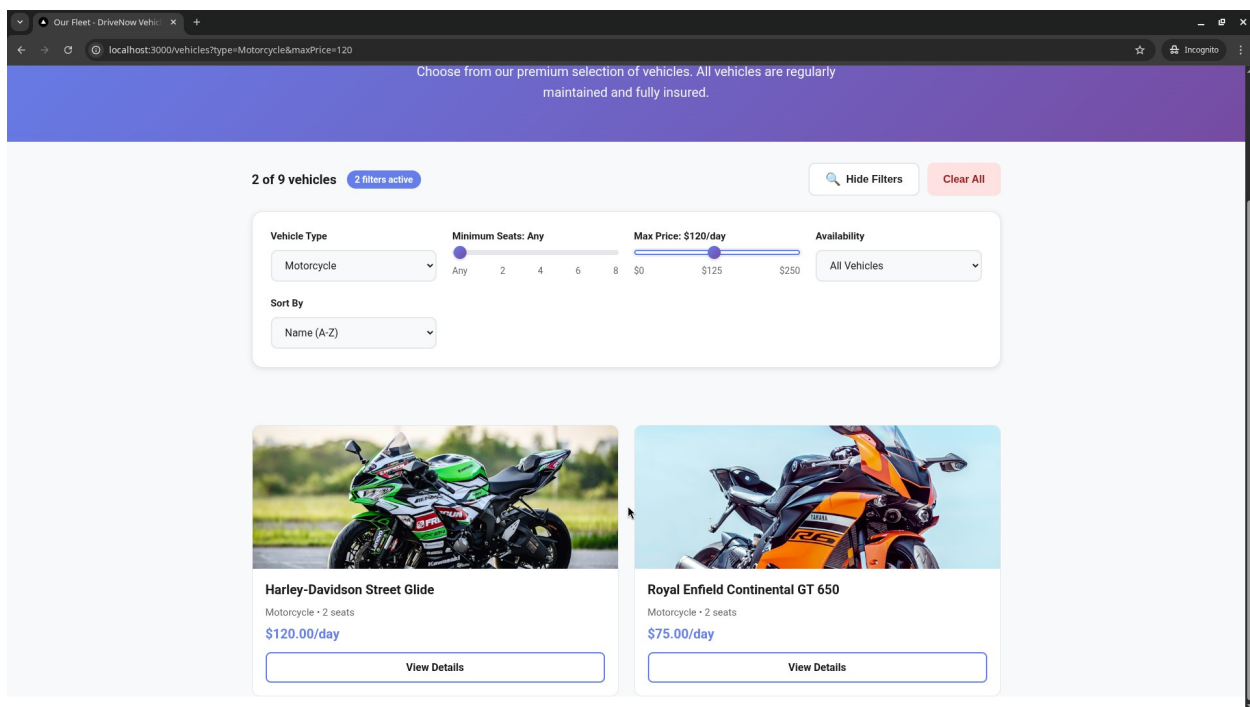
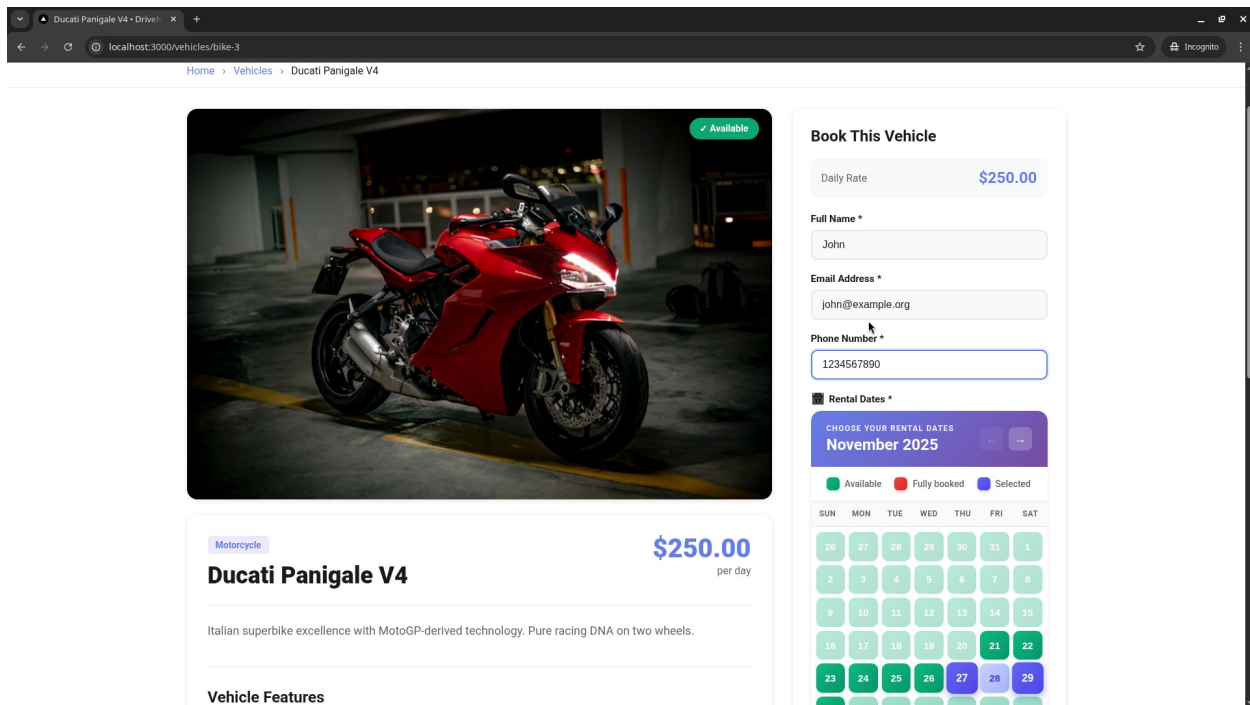
6. Initialize the database:

- o Go to <http://localhost:3000/admin>
- o Login with admin/admin123
- o Click “Initialize Database” to create tables and sample data

1How It Works

User Flow

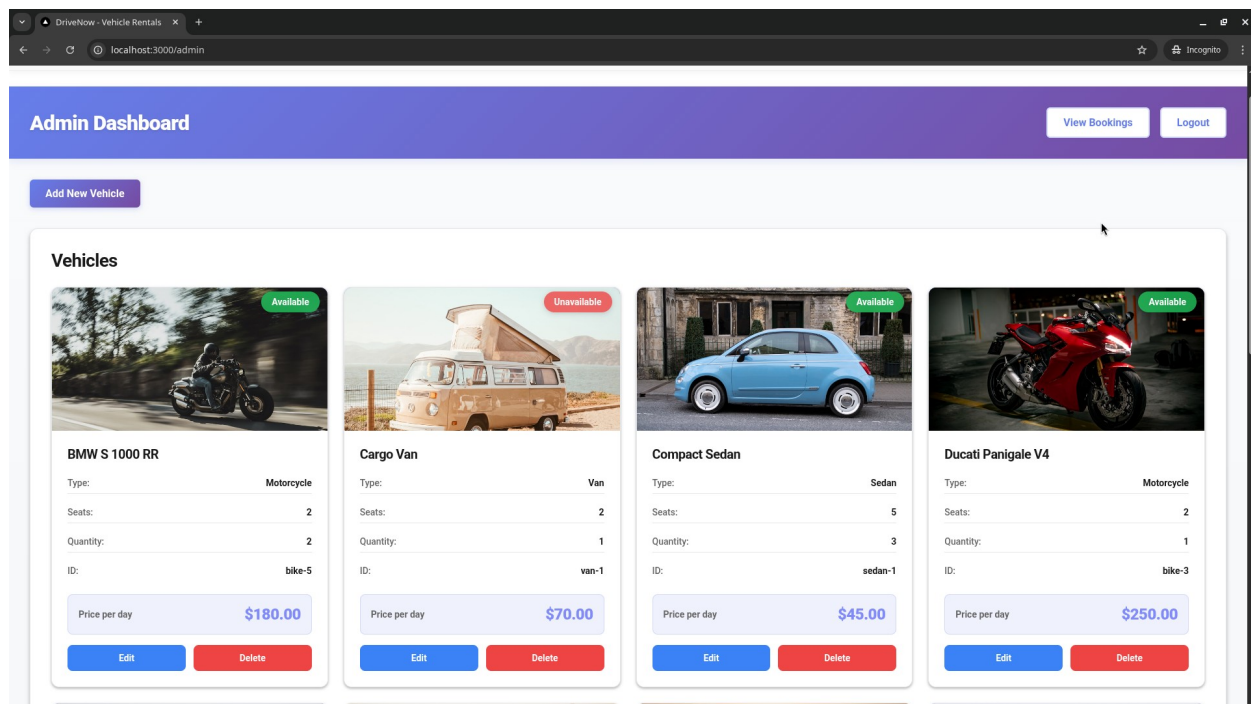


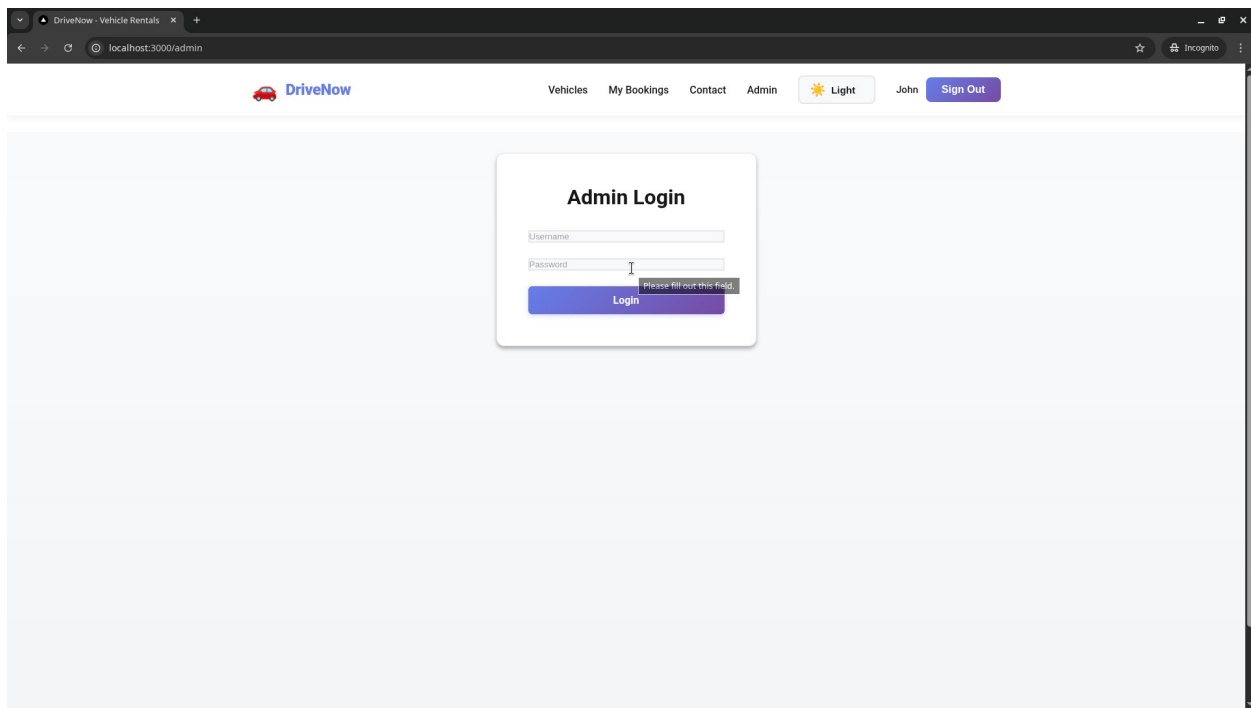


Browsing & Booking (Customers)

1. User visits homepage and clicks “Browse Vehicles”
2. Sees all vehicles with filters (type, price, seats)
3. Clicks on a vehicle to see details

4. Needs to login/signup to make a booking
5. Fills in dates and confirms booking
6. Can view all their bookings in the Bookings page





*A
d
m
i
n*

Management

1. Admin visits /admin and logs in
2. Views all vehicles in a table format
3. Can add new vehicles with details
4. Can edit existing vehicle information
5. Can toggle vehicle availability
6. Can delete vehicles from the system

Data Flow

User Request (Browser)



Next.js Page Component



API Route Handler



Database Query



MySQL Database



Returns Result



JSON Response → React Component → Display to User

Database Schema

The system uses 5 main tables:

```
CREATE TABLE IF NOT EXISTS vehicles (  
  id VARCHAR(255) PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  type VARCHAR(100) NOT NULL,  
  seats INT NOT NULL,  
  pricePerDay DECIMAL(10, 2) NOT NULL,  
  image VARCHAR(500),  
  description TEXT,  
  available BOOLEAN DEFAULT true,  
  quantity INT NOT NULL DEFAULT 1,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,      You, 7 days ago • fix vehi  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
)
```

```
CREATE TABLE IF NOT EXISTS users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  phone VARCHAR(50),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
)
```

```
CREATE TABLE IF NOT EXISTS bookings (
  id INT AUTO_INCREMENT PRIMARY KEY,
  vehicle_id VARCHAR(255) NOT NULL,
  vehicle_name VARCHAR(255) NOT NULL,
  user_id INT,
  customer_name VARCHAR(255) NOT NULL,
  customer_email VARCHAR(255) NOT NULL,
  customer_phone VARCHAR(50) NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  days INT NOT NULL,
  total_price DECIMAL(10, 2) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (vehicle_id) REFERENCES vehicles(id) ON DELETE CASCADE,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
)
```

```
CREATE TABLE IF NOT EXISTS sessions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  session_token VARCHAR(255) UNIQUE NOT NULL,
  expires_at TIMESTAMP NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  INDEX idx_session_token (session_token),
  INDEX idx_expires_at (expires_at)
)
```

1. Vehicles Table

Stores all vehicle information.

Key Queries:

- **Get all vehicles:**

Used in: Vehicles listing page

- **Get filtered vehicles:**

Used in: Vehicle filter component with dynamic parameters

- **Get single vehicle:**

Used in: Vehicle detail page

- **Create vehicle:**

Used in: Admin panel - Add Vehicle

- **Update vehicle:**

Used in: Admin panel - Edit Vehicle

- **Delete vehicle:**

Used in: Admin panel - Delete Vehicle

2. Users Table

Stores customer account information.

Key Queries:

- **Check if email exists:**

Used in: Signup validation

- **Get user by email:**

Used in: Login authentication

- **Create new user:**

Used in: User signup

3. Sessions Table

Manages user login sessions (who is logged in).

Key Queries:

- **Verify user session:**

Used in: Every protected API route (bookings, user profile)

- **Create new session:**

Used in: Login endpoint

- **Clean up expired sessions:**

Used in: Maintenance (manual cleanup)

4. Bookings Table

Stores all vehicle rental reservations.

Key Queries:

- **Get user's bookings:**

Used in: User bookings page

- **Check vehicle availability:**

Used in: Booking validation (count overlapping bookings)

- **Create new booking:**

Used in: Booking creation endpoint

- **Get booking details:**

Used in: Booking confirmation page

5. Admin Users Table

Stores admin credentials.

Key Query:

- **Admin login:**

Used in: Admin panel authentication

Key Features Explained

Authentication

The app uses **session-based authentication** with HTTP-only cookies:

1. User logs in with email and password
2. Password is hashed using PBKDF2 (never stored as plain text)
3. A session token is created and stored in the database

4. Token is sent to browser as an HTTP-only cookie
5. On next request, token is verified against database
6. Session expires after 30 days

Security features: - Passwords hashed with salt (PBKDF2) - Session tokens hashed before storage - HTTP-only cookies (can't be accessed by JavaScript) - Sessions automatically expire

Vehicle Filtering

When users filter vehicles, the system:

1. Reads filter parameters from URL (type, price, seats, etc.)
2. Builds a SQL WHERE clause dynamically
3. Applies sorting (by name, price, or seats)
4. Returns matching vehicles

Supported filters: - Vehicle type (Sedan, SUV, Van, etc.) - Minimum seats (1+) - Maximum price per day - Availability (available/unavailable) - Sorting options (name, price, seats)

Booking System

When a customer books a vehicle:

1. Checks if user is logged in
2. Verifies vehicle availability for requested dates
3. Counts overlapping bookings
4. Compares against vehicle quantity
5. If space available, creates booking record
6. Calculates total price = daily rate × number of days

Availability logic: - Each vehicle has a quantity (e.g., 3 Sedans available) - Bookings check for overlapping date ranges - Can accept bookings if quantity > booked count for those dates

API Endpoints

Public Endpoints (No Login Required)

GET /api/vehicles - Returns all vehicles or filtered results - Accepts query parameters: type, minSeats, maxPrice, availability, sortBy

GET /api/vehicles/[id] - Returns details of a single vehicle

POST /api/auth/signup - Create new user account - Body: { name, email, password, phone }

POST /api/auth/login - User login - Body: { email, password }

GET /api/auth/session - Check if user is logged in - Returns current user info

Protected Endpoints (Login Required)

GET /api/bookings - Get current user's bookings - Requires valid session

POST /api/bookings - Create new booking - Body: { vehicleId, startDate, endDate, ... }

PUT /api/bookings/[id] - Update booking details

DELETE /api/bookings/[id] - Cancel booking

Admin Endpoints

POST /api/admin/auth - Admin login - Body: { username, password }

POST /api/vehicles - Admin: Create new vehicle - Body: { id, name, type, seats, pricePerDay, image, description, available, quantity }

PUT /api/vehicles/[id] - Admin: Update vehicle details

DELETE /api/vehicles/[id] - Admin: Delete vehicle

POST /api/init-db - Admin: Initialize database with tables and sample data

Key Components

VehicleFilter Component

- Displays filter options (type, price, seats, availability)
- Updates URL parameters on filter change

- Used in: Vehicles page

How it works: 1. Receives all available vehicles 2. Extracts unique types and max price 3. Shows filter sliders and dropdowns 4. Updates URL on change 5. Displays filtered results below

VehicleCard Component

- Shows single vehicle card with image, name, and price
- Used in: Vehicle list

BookingForm Component

- Date picker to select rental dates
- Calculates number of days and total price
- Submits booking to API
- Shows confirmation message
- Used in: Vehicle detail page

Header Component

- Navigation menu (Homepage, Vehicles, Bookings, Admin)
- Shows login status
- Logout button for logged-in users

ThemeToggle Component

- Switches between light and dark mode
- Stores preference in browser

AuthContext

- Provides authentication state to entire app
- Tracks logged-in user
- Manages session

Environment Variables

Create a .env.local file with these variables:

```
# Database
DB_HOST=localhost      # MySQL server address
DB_PORT=3306          # MySQL port
DB_USER=root          # MySQL username
DB_PASSWORD=your_password # MySQL password
DB_NAME=vehicle_rental # Database name

# Admin
ADMIN_USERNAME=admin    # Admin panel username
ADMIN_PASSWORD=admin123 # Admin panel password
```

Running the Application

Development Mode

- Starts Next.js development server on `http://localhost:3000`
- Hot reload enabled (changes apply instantly)

Production Build

- Optimized production build
- Better performance and smaller bundle size

Linting

- Checks code for errors and style issues
- Uses ESLint for code quality

Troubleshooting

Database Connection Error

- Check MySQL is running
- Verify credentials in `.env.local`
- Ensure database `vehicle_rental` exists

Admin Login Fails

- Verify `ADMIN_USERNAME` and `ADMIN_PASSWORD` in `.env.local`
- Clear browser cache/cookies
- Try incognito mode

Bookings Not Showing

- Make sure you're logged in
- Check that bookings exist in database
- Verify session hasn't expired (30 days)

Images Not Displaying

- Check image path is correct
- Ensure file exists in /public/images/
- Verify image URL in database

Future Enhancements

- Email notifications for bookings
- Payment integration
- Advanced filtering and search
- User profile management
- Booking modifications
- Vehicle maintenance tracking
- Customer reviews and ratings
- Multi-language support

Security Considerations

- Always keep .env.local secret (never commit to git)
- Use strong admin password in production
- Regularly backup MySQL database
- Update Node.js packages regularly
- Enable HTTPS in production
- Use environment-specific configs
- Validate all user inputs